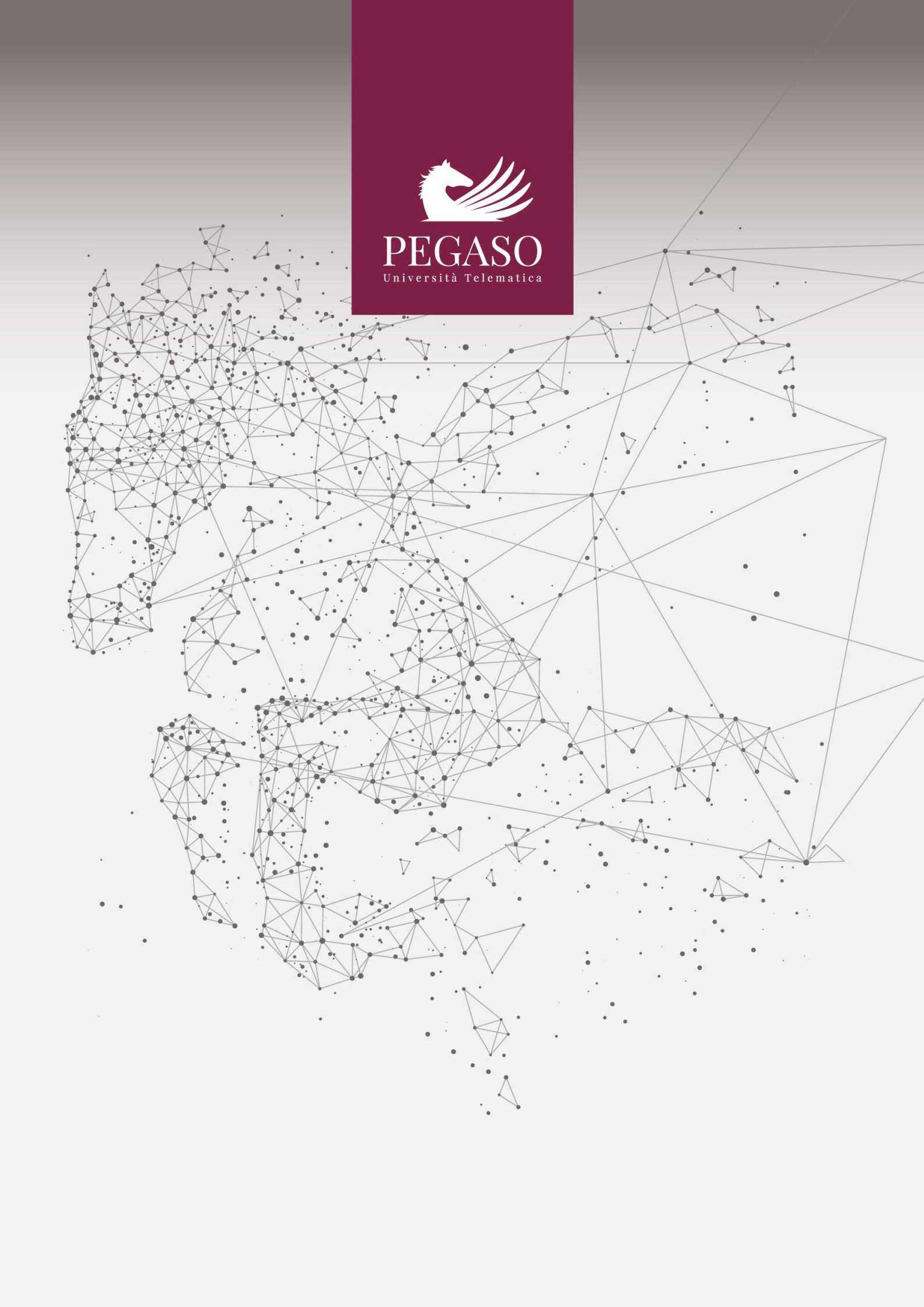

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTERFACCE UTENTE 3

2. X WINDOW SYSTEM 5

3. THIN CLIENT 7

BIBLIOGRAFIA 9

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Interfacce utente

La responsabilità primaria delle macchine client è quella di offrire agli utenti i mezzi con cui possono comunicare con i server situati altrove nella rete. Ci sono due diversi approcci da adottare che supportano questa interazione. In primo luogo, esiste una controparte separata per ogni servizio remoto sul computer client. Questa controparte è in grado di comunicare con il servizio sulla rete. Un calendario che viene eseguito sullo smartphone di un utente e deve essere sincronizzato con un calendario remoto che può essere condiviso è un tipico scenario di esempio. In questo caso particolare, la sincronizzazione verrà gestita da un protocollo a livello di applicazione, come illustrato nella Figura 1.

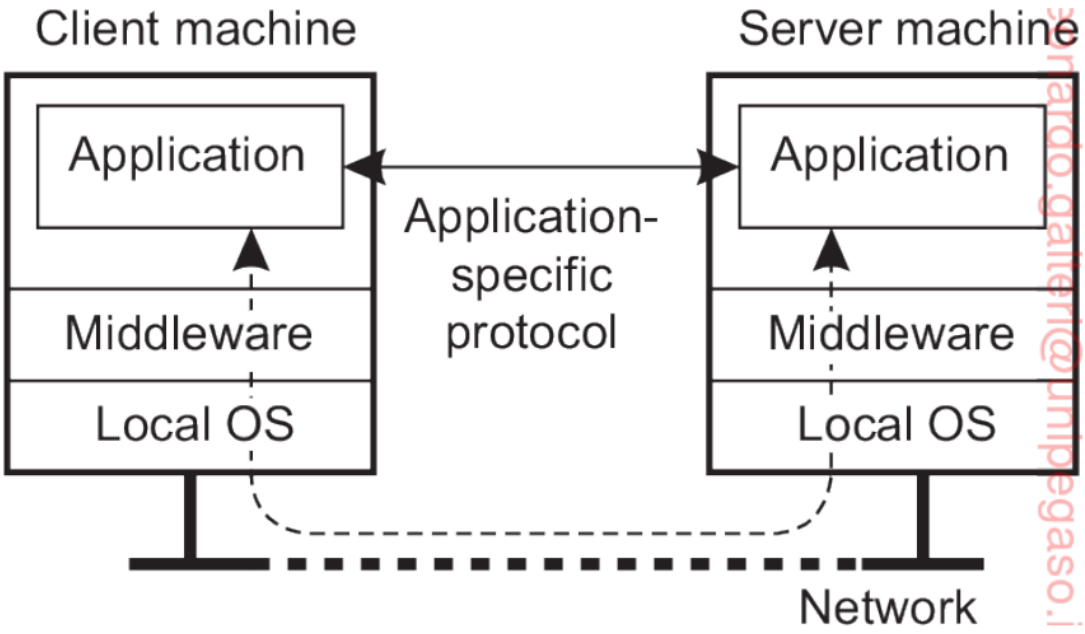


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Una seconda opzione è quella di offrire semplicemente un'interfaccia utente semplice e diretta per facilitare l'accesso diretto ai servizi remoti. Per questo motivo, la macchina client funge solo da terminale e non è necessario disporre di alcuna memoria locale. Ciò si traduce in una soluzione non specifica per una particolare applicazione, come illustrato nella Figura 2. Quando si tratta di interfacce utente connesse a una rete, tutti i dati sono elaborati e salvati sul server.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

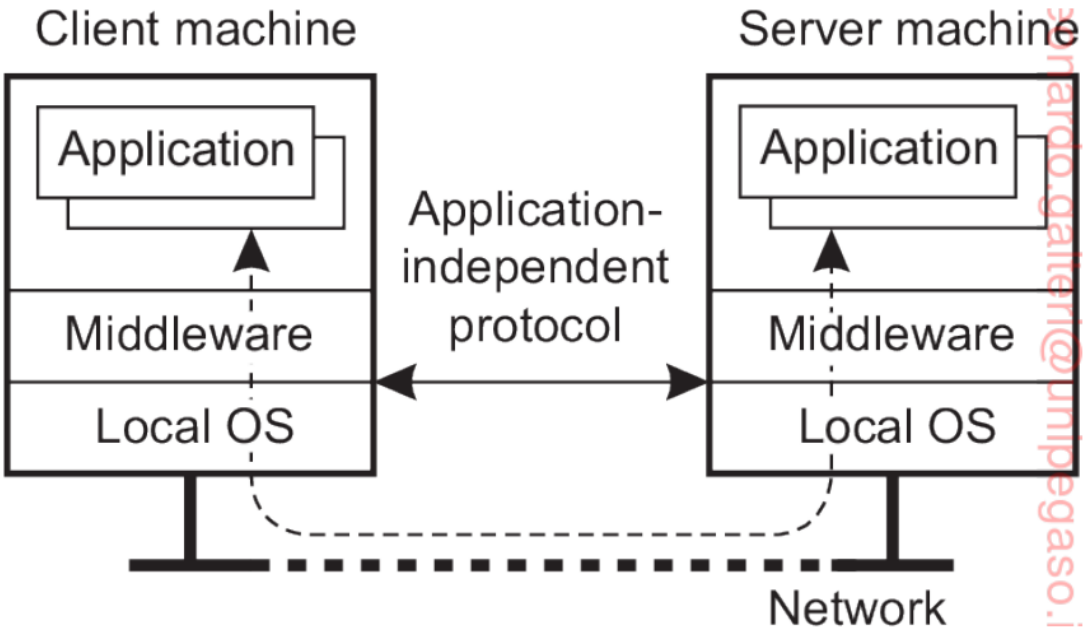


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

L'aumento della connettività ad Internet e la popolarità dei dispositivi elettronici portatili hanno portato molta attenzione all'approccio thin-client. Le soluzioni thin-client stanno diventando sempre più popolari grazie al fatto che semplificano il processo di gestione del sistema.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. X Window System

X Window System (noto in anche come X Window, X11 o semplicemente come X) è una delle interfacce utente di rete più longeve e anche più utilizzate oggi. È stato sviluppato nel 1980. I terminali sono controllati da X Window System e consistono tipicamente in un monitor, una tastiera e un dispositivo di puntamento come un mouse. X è in grado di supportare non solo terminali convenzionali, come quelli che si trovano su computer desktop e workstation, ma anche dispositivi contemporanei, come i touchscreen presenti su tablet e smartphone. Un modo per pensare a X è come un componente di un sistema operativo responsabile del funzionamento del terminale. Il componente a cui ci riferiremo come kernel X fungerà da unità di elaborazione centrale (CPU) del sistema.

Come risultato del fatto che contiene tutti i driver di periferica specifici del terminale, è generalmente molto dipendente dall'hardware.

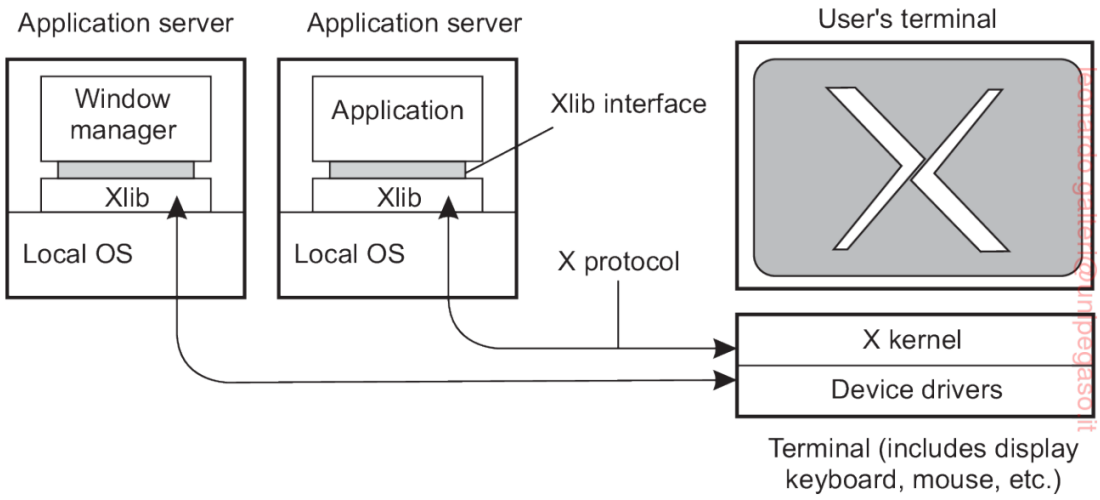


Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Il kernel X fornisce un'interfaccia utente che si trova all'estremità inferiore in una ipotetica scala di complessità, e può essere usato non solo per controllare lo schermo, ma anche per registrare eventi dalla tastiera e dal mouse. Una libreria nota come Xlib rende questa interfaccia disponibile alle applicazioni in modo che possano utilizzarla. La Figura 3 illustra questa struttura generale per l'organizzazione. È

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

importante tenere presente che le applicazioni non utilizzano quasi mai Xlib direttamente; piuttosto, distribuiscono toolkit più user-friendly che vengono implementati su Xlib.

Il fatto che il kernel X e le applicazioni X non debbano essere ospitati sullo stesso computer è un aspetto intrigante di X. In particolare, X fornisce ciò che è noto come protocollo X. Questo è un protocollo di comunicazione che opera a livello di applicazione e consente a un'istanza di Xlib di scambiare dati ed eventi con un kernel X. Per esempio, Xlib è in grado di inviare richieste al kernel X per una varietà di scopi, tra cui la creazione di nuove finestre, la distruzione di finestre esistenti, l'impostazione dei colori e la definizione del tipo di cursore che verrà visualizzato.

In risposta, il kernel X reagirà agli eventi locali come l'input dalla tastiera e dal mouse inviando i pacchetti di eventi a Xlib.

Alcune applicazioni possono comunicare con il kernel X contemporaneamente. Un programma in particolare, noto come window manager, ottiene privilegi elevati. Questa applicazione ha la capacità di controllare come il display appare all'utente dettando il suo "look and feel", ovvero ha la capacità di dettare come ogni finestra deve essere abbellita con pulsanti aggiuntivi, come le finestre devono essere disposte sul display, e così via. Altre applicazioni dovranno seguire queste regole alla lettera. In realtà, ciò significa che una parte significativa dell'interazione che normalmente avviene tra un'applicazione e un terminale X viene invece reindirizzata attraverso un window manager.

È interessante pensare a come il modello client-server funzioni effettivamente con il sistema X window. Dovrebbe essere ovvio, sulla base di ciò che abbiamo discusso fino a questo punto, che il kernel X è il destinatario delle richieste di manipolare la visualizzazione. Riceve queste richieste da applicazioni, possibilmente situate in altre posizioni. Se visto in questa luce, il kernel X svolge la funzione di un server, mentre le applicazioni svolgono la funzione di client. X ha deciso di utilizzare questa terminologia, sebbene tecnicamente parlando sia accurata, può facilmente portare a confusione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Thin client

Ovviamente, le applicazioni manipolano un display utilizzando i particolari comandi di visualizzazione resi disponibili da X. Nella maggior parte dei casi, questi comandi saranno trasmessi sulla rete, dopodiché il kernel X eseguirà i passi necessari per eseguirli. Le applicazioni scritte per X dovrebbero, come ovvio e a causa della natura della piattaforma, separare la logica dell'applicazione dai comandi dell'interfaccia utente. La realtà, tuttavia, è che questo è raramente il caso. Infatti, una parte significativa della logica dell'applicazione e dell'interazione dell'utente sono strettamente accoppiate. Ciò indica che un'applicazione invierà numerose richieste al kernel X, per le quali attenderà una risposta prima di poter procedere alla fase successiva del processo. Quando si lavora su una rete WAN con lunghi tempi di latenza, questo comportamento sincrono può avere un impatto negativo sulle prestazioni.

Questo problema può essere risolto in diversi modi. La prima opzione è quella di riprogettare l'implementazione del protocollo X, in modo simile a come descrive come avviene per NX (NoMachine). Questa soluzione dedica una parte significativa della sua attenzione alla riduzione dei requisiti di larghezza di banda riducendo le dimensioni dei messaggi X. Per raggiungere questo scopo, si fa in modo che i messaggi contengano sia una parte fissa, che funge da identificatore, sia una parte variabile. Esistono molti scenari in cui più messaggi condivideranno lo stesso identificatore, nel qual caso tali messaggi conterranno spesso dati identici tra loro. I messaggi che condividono un identificatore possono essere confrontati utilizzando questa proprietà, che consente agli utenti di inviare solo le differenze tra loro. È molto più semplice implementare la decodifica sul lato ricevente se il mittente e il destinatario tengono traccia dei rispettivi identificatori.

Si possono ottenere riduzioni della larghezza di banda fino a un fattore di 1000, che consente a X di operare anche su collegamenti a bassa larghezza di banda con una capacità massima di 9600 kbps.

In alternativa all'uso di X, ricercatori e professionisti hanno anche esaminato la possibilità di consentire a un'applicazione di avere il controllo completo sul display remoto o di arrivare al livello dei pixel. Successivamente, gli aggiornamenti al bitmap vengono inviati attraverso la rete al display, dove vengono immediatamente applicati al frame buffer locale.

Il Virtual Network Computing (VNC), che esiste dalla fine degli anni 1990, è un esempio ben noto di questo approccio. Consentire all'applicazione di controllare la visualizzazione richiede ovviamente l'utilizzo di metodi di codifica complessi al fine di evitare di creare un collo di bottiglia in termini di quantità di larghezza di banda disponibile. Prendiamo, ad esempio, il caso di visualizzare un flusso video su uno

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

schermo semplice con dimensioni di 320 pixel per 240 pixel a una velocità di 30 fotogrammi al secondo. Se ogni pixel è codificato con 24 bit, allora avremmo bisogno di una larghezza di banda di circa 53 Mbps se non abbiamo uno schema di codifica efficiente. Nella pratica reale, vengono utilizzati una varietà di metodi di codifica, ma la selezione del metodo più appropriato dipende in genere dall'applicazione.

Non è possibile utilizzare la semantica dell'applicazione quando si inviano dati pixel grezzi, al contrario dell'invio di protocolli di livello superiore come X, perché la semantica dell'applicazione viene effettivamente persa a quel livello. Questo è uno degli svantaggi dell'invio di dati pixel grezzi. Esiste tuttavia una tecnica diversa in cui vengono offerti diversi comandi di visualizzazione di alto livello come parte della soluzione, che è chiamata THINC. Questi comandi operano a livello dei driver di periferica video e dipendono quindi dal dispositivo utilizzato. Sono più potenti delle operazioni sui pixel grezzi, ma sono meno potenti di quanto offerto da un protocollo come X. Di conseguenza, i server di visualizzazione possono essere resi molto più semplici, il che è vantaggioso per la quantità di potenza di elaborazione che utilizzano. Allo stesso tempo, le ottimizzazioni dipendenti dall'applicazione possono essere utilizzate per ridurre i requisiti di larghezza di banda e sincronizzazione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Baratto R. A., Nieh J., and Kim L. THINC: A Remote Display Architecture for Thin-Client Computing. In 20th Symposium on Operating System Principles, 2005.
- Lai A. and Nieh J. Limits of Wide-Area Thin-Client Computing. In International Conference on Measurements and Modeling of Computer Systems, 2002.
- Pinzari G. NX X Protocol Compression. Technical Report D-309/3-NXP-DOC, NoMachine, 2003.
- Richardson T., Stafford-Fraser Q., Wood K. R., and Hopper A. Virtual NetworkComputing. IEEE Internet Computing, 1998.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).